AUTOMATIZACIÓN DE CONTROL DE ACCESOS MEDIANTE PYTHON

Descripción del proyecto

El acceso a contenidos protegidos en mi organización se rige por un listado de direcciones IP permitidas contenido en allow_list.txt. Con el fin de mantener esta lista actualizada, programé un algoritmo que identifica y remueve automáticamente aquellas IPs que han perdido sus privilegios de acceso según una lista de eliminación establecida.

Abre el archivo que contiene la lista de permisos

Para la primera parte del algoritmo, abrí el archivo "allow_list.txt". Primero, asigné este nombre de archivo como cadena a la variable import_file:

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"
```

Luego, usé una instrucción with para abrir el archivo:

```
# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:
```

En mi algoritmo, la instrucción con se usa junto con la función. open() en modo lectura para abrir el archivo de lista de permisos con el propósito de leerlo. El propósito de abrir el archivo es permitirme acceder a las direcciones IP almacenadas en el archivo de la lista de permisos. La palabra clave with ayudará a gestionar los recursos cerrando el archivo tras salir de la instrucción with. En el código con with open(import_file, "r") as file:, la función open() tiene dos parámetros. La primera identifica el archivo a importar, y luego la segunda indica lo que quiero hacer con el archivo. En este caso, la "r" indica que quiero leerla. El código también utiliza la palabra clave as para asignar una variable llamada file. file almacena la salida de la función .open() mientras trabajo dentro de la instrucción with.

Lee el contenido del archivo

Para leer el contenido del archivo, usé el método .read() para convertirlo en la cadena.

```
with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()
```

Cuando uso una función .open() que incluye el argumento "r" para "read", puedo llamar a la función .read() en el cuerpo de la sentencia with. El método .read() convierte el archivo en una cadena y me permite leerlo. Apliqué el método .read() a la variable file identificada en la instrucción with. Luego, asigné la salida de la cadena de este método a la variable ip_addresses.

En resumen, este código lee el contenido del archivo "allow_list.txt" en un formato de cadena que me permite usar esa cadena para organizar y extraer datos en mi programa de Python.

Convierte la cadena en una lista

Para eliminar direcciones IP individuales de la lista de permisos, necesitaba que estuviera en formato de lista. Por lo tanto, usé el método .split() para convertir la cadena de ip_addresses en una lista:

```
# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()
```

La función .split() se llama añadiéndola a una variable de cadena. Funciona convirtiendo el contenido de una cadena en una lista. El propósito de dividir ip_addresses en una lista es facilitar la eliminación de direcciones IP de la lista de permisos. Por defecto, la función. split() divide el texto por espacios en blanco en elementos de lista. En este algoritmo, la función. split() toma los datos almacenados en la variable ip_addresses, que es una cadena de direcciones IP separadas por un espacio en blanco, y convierte esta cadena en una lista de direcciones IP. Para almacenar esta lista, la reasigné a la variable ip_addresses.

Iterar por la lista de eliminación

Una parte clave de mi algoritmo consiste en iterar por las direcciones IP que son elementos en el remove_list. Para ello, incorporé un bucle for:

```python
# Build iterative statement
# Name loop variable `element`
# Loop through `remove_list`

for element in remove_list:
```

El bucle for en Python repite el código para una secuencia especificada. El propósito general del bucle for en un algoritmo de Python como este es aplicar sentencias de código específicas a todos los elementos de una secuencia. La palabra clave for inicia el bucle for. Le siguen el element de la variable bucle y la palabra clave in. La palabra clave in. indica iterar a través de la secuencia ip_addresses y asignar cada valor al elemento de la variable de bucle.

Eliminar direcciones IP que estén en la lista de eliminación

Mi algoritmo requiere eliminar cualquier dirección IP de la lista de permisos, ip_addresses, que también esté contenida en remove_list. Como no había duplicados en ip_addresses, pude usar el siguiente código para hacerlo:

```python
for element in remove_list:

  # Create conditional statement to evaluate if `element` is in `ip_addresses`

    if element in ip_addresses:

      # use the `.remove()` method to remove
      # elements from `ip_addresses`

        ip_addresses.remove(element)
```

Primero, dentro de mi bucle for, creé un condicional que evaluaba si el elemento de la variable bucle se encontraba o no en la lista de ip_addresses. Hice esto porque aplicar .remove() a elementos que no se encontraban en ip_addresses daba lugar a un error.

Luego, dentro de esa condicional, apliqué. remove() a ip_addresses. Pasé el elemento de variable de bucle como argumento para que cada dirección IP que estuviera en el remove_list fuera eliminada de ip_addresses.

Actualiza el archivo con la lista revisada de direcciones IP

Como paso final en mi algoritmo, necesitaba actualizar el archivo de la lista de permisos con la lista revisada de direcciones IP. Para ello, primero necesitaba convertir la lista de nuevo en una cadena. Usé el método .join() para esto:

```
# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = "\n".join(ip_addresses)
```

El método .join() combina todos los elementos de un iterable en una cadena. El método .join() se aplica a una cadena que contiene caracteres que separarán los elementos del iterable una vez unidos en una cadena. En este algoritmo, usé el método .join() para crear una cadena a partir del ip_addresses de lista para poder pasarla como argumento al método .write() al escribir en el archivo "allow_list.txt". Usé la cadena ("\n") como separador para indicar a Python que colocara cada elemento en una nueva línea.

Luego, usé otra instrucción with y el método .write() para actualizar el archivo:

```
# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:

  # Rewrite the file, replacing its contents with `ip_addresses`

  file.write(ip_addresses)
```

Esta vez, usé un segundo argumento de "w" con la función open() en mi sentencia with. Este argumento indica que quiero abrir un archivo para escribir sobre su contenido. Al usar este argumento "w", puedo llamar a la función .write() en el cuerpo de la sentencia with. La función . write() escribe los datos de cadenas en un archivo especificado y reemplaza cualquier contenido existente.

En este caso quería escribir la lista de permisos actualizada como una cadena en el archivo "allow_list.txt". De este modo, el contenido restringido dejará de ser accesible a ninguna dirección IP que haya sido eliminada de la lista de permisos. Para reescribir el archivo, aplique la función. write() al archivo objeto del file que identifiqué en la instrucción with. Introduje la variable ip_addresses como argumento para especificar que el contenido del archivo especificado en la instrucción con debe ser reemplazado por los datos de esta variable.

Resumen

He creado un algoritmo que elimina direcciones IP identificadas en una variable remove_list del archivo "allow_list.txt" de direcciones IP aprobadas. Este algoritmo consistía en abrir el archivo, convertirlo en una cadena para ser leída y luego convertir esa cadena en una lista almacenada en la variable

ip_addresses. Luego iteré las direcciones IP en remove_list. Con cada iteración, evaluaba si el elemento formaba parte de la lista de ip_addresses. Si lo estaba, aplicaba el método .remove() para eliminar el elemento de ip_addresses. Después de esto, utilicé el método .join() para convertir el ip_addresses de nuevo en una cadena de cadena, de modo que podía escribir sobre el contenido del archivo "allow_list.txt" con la lista revisada de direcciones IP.

Nota: *Los comentarios y nombres de variables dentro del algoritmo de Python se encuentran en inglés debido a que este proyecto se desarrolló como parte de una actividad académica. El entorno de programación utilizado fue* ***Jupyter Notebook.***